

CESIZen

Plateforme de bien-être numérique

DOSSIER — BLOC 3

Sécuriser, déployer et maintenir les applications

API Spring Boot · Front Next.js · Mobile Flutter

Version 1.0 — Juillet 2026

Sommaire

Partie I — Plan de sécurisation	4
1. Objet et périmètre	4
2. Architecture et flux de données (DFD)	4
3. Couverture OWASP Top 10 (2021)	5
4. Modélisation des menaces	6
5. Surface d'exposition — gestion des ports	7
6. Droits sur les services et les conteneurs	7
7. Analyse statique (SAST) — SonarCloud	8
8. Mesures détaillées par application	8
9. Sécurité de la chaîne de livraison (CI/CD)	9
10. Conformité RGPD	9
11. Gestion de crise — réponse à incident	11
12. Risques résiduels	11
13. Conclusion	11
Partie II — Plan de déploiement	12
1. Objet	12
2. Gestion du code source	12
3. Intégration et déploiement continu (CI/CD)	14
4. Déploiement par application	16
5. Environnements	16
6. Supervision du déploiement	18
7. Procédure de mise en production	18
8. Conclusion	18
Partie III — Dossier de maintenance	19
1. Objet	19
2. Supervision et monitoring	19
3. Gestion des tickets (GitHub Issues)	19
4. Maintenance préventive — mises à jour	21
5. Veille technologique	21
6. Procédure de réponse à incident	21
7. Types de maintenance et engagements	21
8. Conclusion	22
Partie IV — Veille technologique	23
Méthodologie	23
Fiche 1 — Vulnérabilités Jackson (CVE-2026-54512 / 54513)	24
Fiche 2 — Spring Boot 4 & Spring Security 6	25
Fiche 3 — Vulnérabilité Next.js 16 (contournement de middleware)	26

Synthèse26

Partie I — Plan de sécurisation

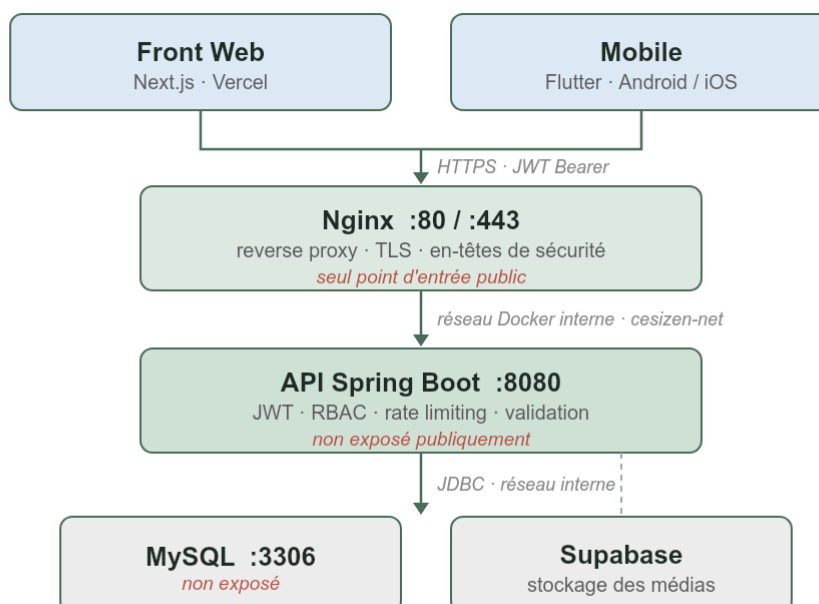
1. Objet et périmètre

Ce document décrit les mesures de sécurité mises en œuvre sur l'ensemble des composants de la plateforme CESIZen, ainsi que les risques résiduels identifiés. Il couvre les **trois applications** : API REST (Spring Boot 4 / Java 21), front web (Next.js 16 sur Vercel) et application mobile (Flutter).

La démarche suit le référentiel **OWASP Top 10 (2021)** et applique le principe de **sécurité par conception** : blocage par défaut, ouverture explicite du strict nécessaire.

2. Architecture et flux de données (DFD)

Le schéma ci-dessous représente les flux de données entre les clients, le point d'entrée public et les services internes.



Frontières de confiance. Tout ce qui vient du client est non fiable : validation systématique côté API. Nginx est la seule surface exposée à Internet ; l'API et MySQL ne sont joignables que depuis le réseau Docker interne.

3. Couverture OWASP Top 10 (2021)

#	Catégorie	Mesures mises en place
A01	Broken Access Control	JWT obligatoire hors routes publiques ; @PreAuthorize par rôle ; comptes désactivables.
A02	Cryptographic Failures	BCrypt (force 10) ; JWT signé HMAC-SHA256, secret en variable d'environnement ; HTTPS + HSTS.
A03	Injection	Spring Data JPA (requêtes paramétrées) ; validation Jakarta @Valid sur tous les DTO.
A04	Insecure Design	Moindre privilège (user MySQL dédié) ; API stateless ; réponse neutre sur mot de passe oublié.
A05	Security Misconfiguration	CSRF désactivé (REST stateless) ; Swagger désactivé en prod ; en-têtes HTTP de sécurité.
A06	Vulnerable Components	Dependabot (3 écosystèmes) ; Trivy à chaque CI ; CVE Jackson, Tomcat et Next.js corrigées.
A07	Auth Failures	Rate limiting bucket4j ; tokens de reset à usage unique ; journalisation des connexions.
A08	Software/Data Integrity	CI avec Gitleaks + Trivy ; images Docker sur GHCR ; Dependabot sur les actions GitHub.
A09	Logging & Monitoring	log_connexion, log_activite (audit AOP) ; Prometheus + Grafana + Loki.
A10	SSRF	Aucun endpoint n'appelle une URL fournie par l'utilisateur ; SMTP configuré statiquement.

4. Modélisation des menaces

Méthode de cotation. Chaque menace est cotée sur deux axes — la **probabilité** (Faible = 1, Moyenne = 2, Élevée = 3) et l'**impact** (Faible = 1, Moyen = 2, Élevé = 3, Critique = 4). La **criticité** est le produit des deux (Probabilité × Impact), puis classée selon la matrice ci-dessous. Elle détermine l'ordre de priorité des actions correctives.

Matrice de criticité (Probabilité × Impact).

Prob. \ Impact	Faible (1)	Moyen (2)	Élevé (3)	Critique (4)
Élevée (3)	Moyenne (3)	Élevée (6)	Critique (9)	Critique (12)
Moyenne (2)	Faible (2)	Moyenne (4)	Élevée (6)	Élevée (8)
Faible (1)	Faible (1)	Faible (2)	Moyenne (3)	Moyenne (4)

Seuils : Faible = 1-2 · Moyenne = 3-4 · Élevée = 5-8 · Critique = 9-12.

Analyse des menaces principales. Criticité calculée et mesure d'atténuation en place pour chacune :

Menace	Surface	Prob.	Impact	Criticité	Mitigation
Brute-force sur le login	POST /api/auth/login	Moyenne	Élevé	Élevée (6)	Rate limiting 5/min/IP → 429 ; journalisation des échecs
Énumération de comptes	POST /forgot-password	Moyenne	Moyen	Moyenne (4)	Réponse 200 systématique ; rate limiting 3/10 min
Vol de token JWT	En-tête Authorization	Faible	Élevé	Moyenne (3)	HTTPS + HSTS ; expiration ; rotation du secret
Injection SQL	Tous endpoints	Faible	Critique	Moyenne (4)	JPA paramétré ; user BDD sans droits DDL
IDOR	/api/user/{id}	Moyenne	Élevé	Élevée (6)	@PreAuthorize ; vérification de propriété
XSS stocké	Front web	Faible	Moyen	Faible (2)	React échappe par défaut ; CSP
Clickjacking	Front web	Faible	Faible	Faible (1)	X-Frame-Options: DENY ; frame-ancestors 'none'
Fuite de secret	Historique git	Moyenne	Critique	Élevée (8)	Gitleaks + hook pre-commit ; secrets en variables d'env
Dépendance vulnérable	Toutes les apps	Élevée	Élevé	Critique (9)	Trivy bloquant ; Dependabot ; OWASP Dependency Check
Exposition service d'admin	phpMyAdmin, Grafana	Moyenne	Élevé	Élevée (6)	Non déployés en production

Priorisation. La menace la plus critique — dépendance vulnérable (9) — justifie le scan bloquant Trivy et Dependabot ; les fuites de secret (8) et les accès non autorisés (brute-force, IDOR, admin, à 6) sont traités en priorité. Les risques faibles (XSS, clickjacking) sont couverts par des mesures standard sans effort dédié.

5. Surface d'exposition — gestion des ports

Principe : n'exposer que l'indispensable. Un seul port public en production.

Service	Port	Dev	Prod	Accessible par
Nginx	80 / 443	8080→80	Oui	Public — seul point d'entrée
API Spring Boot	8080	interne	interne	Nginx uniquement
MySQL	3306	exposé	NON (ports: !reset [])	API uniquement
phpMyAdmin	8082	exposé	absent	Admin local
Mailhog (SMTP test)	1025 / 8025	exposé	absent	Développement
Prometheus	9090	stack séparée	interne	Grafana
Grafana	3001	stack séparée	à protéger	Admin local
Loki	3100	stack séparée	interne	Promptail / Grafana

En production, `docker-compose.prod.yml` supprime explicitement les ports de MySQL et n'inclut ni phpMyAdmin ni Mailhog. Seul Nginx écoute sur l'extérieur.

6. Droits sur les services et les conteneurs

Conteneur non-root. Le Dockerfile de l'API crée un utilisateur système dédié (`adduser -S -G cesizen cesizen`) et bascule dessus (`USER cesizen`). Une évaison applicative ne donne pas les droits root dans le conteneur.

Réseau Docker nommé. Tous les services partagent un bridge dédié `cesizen-net`. MySQL et l'API ne sont pas publiés sur l'hôte : seule la résolution DNS interne permet de les joindre.

Utilisateur MySQL à droits minimaux. En production, `cesizen_app` est privé des droits DDL (`REVOKE CREATE, ALTER, DROP, INDEX, REFERENCES`) via un script d'init. Ne subsistent que `SELECT, INSERT, UPDATE, DELETE`. Limite connue : le script ne s'exécute qu'au premier démarrage, sur un volume vide.

Fichiers uploadés. Taille limitée à 10 Mo. Le type MIME réel n'est pas vérifié — identifié comme risque résiduel.

7. Analyse statique (SAST) — SonarCloud

Le dépôt `api_zen` est lié à SonarCloud. L'analyse s'exécute dans le pipeline d'intégration continue, après compilation et tests, avec transmission du rapport de couverture JaCoCo. Sonar contrôle bugs, vulnérabilités, code smells et duplication.

Le step est gardé par `if: env.SONAR_TOKEN != ''` : en l'absence de jeton (pull request Dependabot par exemple), il est sauté sans faire échouer la CI. Les jetons `SONAR_TOKEN`, `SONAR_ORGANIZATION` et `SONAR_PROJECT_KEY` sont stockés dans les secrets GitHub Actions, jamais dans le code.

8. Mesures détaillées par application

8.1 API — Spring Boot

Rate limiting (bucket4j). Filtre `OncePerRequestFilter` par IP, réponse HTTP 429 :

Endpoint	Limite	Protection
POST /api/auth/login	5 / minute	Brute-force
POST /api/auth/forgot-password	3 / 10 minutes	Énumération de comptes, spam d'e-mails

En-têtes HTTP. Posés à deux niveaux (Spring Security + nginx) : X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Content-Security-Policy, Permissions-Policy, et HSTS en production.

Secrets. Le secret JWT a été retiré du code (il figurait en défaut commité) et régénéré. Il ne vit désormais que dans les variables d'environnement et les secrets GitHub. Gitleaks bloque toute réintroduction.

CVE corrigées. jackson-databind (CVE-2026-54512 / 54513) via import de BOM 2.21.4 et 3.1.4 ; Tomcat forcé en 11.0.22 ; Swagger désactivé en production.

8.2 Front — Next.js

En-têtes de sécurité. Six en-têtes dans `next.config.ts`, dont une CSP construite dynamiquement à partir des origines autorisées (API et Supabase uniquement). HSTS émis uniquement en production.

CSP — compromis assumé. La politique autorise `'unsafe-inline'` sur script/style (le App Router injecte des scripts et styles inline). La solution propre — un nonce par requête généré via un middleware — constitue une évolution recommandée.

CVE corrigée. Next.js 16.2.2 portait des CVE HIGH (contournement de middleware), détectées par le scan CI. Corrigées par montée en 16.2.10.

8.3 Mobile — Flutter

Analyse & scan. La CI exécute `flutter analyze` (bloquant) et `flutter test` ; le workflow sécurité scanne pubspec.lock (CVE pub.dev), secrets et misconfigs via Trivy, plus Gitleaks.

Point de vigilance. Le fichier de démonstration contient un mot de passe en clair. Le dépôt étant public, s'assurer qu'aucun compte réel de production ne l'utilise.

9. Sécurité de la chaîne de livraison (CI/CD)

Outil	Rôle	Portée
Trivy	CVE des dépendances, secrets, misconfigs, image Docker	3 apps
Gitleaks	Détection de secrets dans l'historique git	3 apps
OWASP Dependency Check	Scan approfondi base NVD (nocturne)	API
Dependabot	Mises à jour préventives (Maven, npm, pub, actions)	3 apps
SonarCloud	Analyse statique (SAST), couverture	API

La branche `main` de chaque dépôt est protégée par un ruleset : pull request obligatoire et checks de sécurité verts exigés avant tout merge.

Traitement d'une alerte Dependabot. Revue de la PR, vérification de la CI verte, merge. Une montée majeure cassante est fermée et neutralisée par une règle `ignore` si elle récidive.

10. Conformité RGPD

10.1 Données à caractère personnel collectées

Donnée	Finalité	Conservation	Sensible
Nom, prénom	Identification de l'utilisateur	Durée du compte	Non
Adresse e-mail	Authentification, réinitialisation	Durée du compte	Non
Mot de passe (haché BCrypt)	Sécurité de l'accès	Durée du compte	Oui
Date de création du compte	Traçabilité	Durée du compte	Non
IP, horodatage, e-mail tenté	Sécurité, détection brute-force	90 jours (recommandé)	Non
Historique d'activité	Audit des mutations	90 jours (recommandé)	Non
Historique d'exercices	Suivi du bien-être personnel	Durée du compte	OUI — santé

L'historique d'exercices relève des **données de santé** au sens du RGPD : il nécessite un consentement explicite et une protection renforcée.

10.2 Droits des personnes — état d'implémentation

Droit	Mécanisme	État
Accès	GET /api/user/{id} — consultation du profil	Implémenté
Rectification	PUT /api/user/{id}	Implémenté
Effacement	DELETE /api/user/me (auto) ; DELETE /api/user/{id} (anonymisation admin)	Implémenté
Portabilité	Export structuré JSON/CSV	NON implémenté
Opposition / limitation	Désactivation du compte (active = false)	Partiel

10.3 Écarts identifiés

- **Consentement explicite.** Aucun champ de consentement n'est stocké dans le modèle de données de l'API. Or les données de santé exigent un consentement explicite et démontrable. Action requise : ajouter un champ de consentement horodaté à l'inscription.

- **Durée de conservation.** Aucune purge automatique des journaux n'est en place. Les durées indiquées sont des recommandations, pas des mécanismes actifs.
- **Portabilité.** Aucun export structuré n'est proposé.

Violation de données. Notification à la CNIL sous 72 heures et information des personnes concernées si le risque est élevé.

11. Gestion de crise — réponse à incident

- **1. Détection** — alerte Dependabot, échec de scan de sécurité, journaux anormaux, ou signalement utilisateur via une issue GitHub.
- **2. Isolation** — couper le service concerné ; en cas de compromission de token, rotation immédiate du secret JWT (invalide tous les tokens émis).
- **3. Analyse** — examiner log_connexion et log_activite pour délimiter le périmètre et l'impact.
- **4. Correction** — patch, pull request, CI verte, redéploiement.
- **5. Communication** — informer les utilisateurs si des données personnelles sont concernées.
- **6. Obligation RGPD** — notification à la CNIL sous 72 h en cas de violation de données personnelles.

Responsabilité des journaux. L'administrateur de la plateforme est responsable de leur conservation et de leur purge. Rétention recommandée : 90 jours.

12. Risques résiduels

Risque	Gravité	Action recommandée
Consentement RGPD non tracé (données de santé)	Élevée	Champ de consentement horodaté à l'inscription
Aucune purge automatique des journaux	Moyenne	Tâche planifiée de suppression à 90 jours
Droit à la portabilité non implémenté	Moyenne	Endpoint d'export JSON des données utilisateur
CSP avec 'unsafe-inline' (front)	Moyenne	Passer aux nonces via middleware
Upload sans validation MIME réelle	Moyenne	Vérifier les magic bytes côté serveur
Grafana : mot de passe par défaut en prod	Moyenne	Changer GRAFANA_ADMIN_PASSWORD ; tunnel SSH
Rate limiting en mémoire	Faible	Migrer vers bucket4j-redis si haute disponibilité
CVE moderate postcss (toolchain Next)	Faible	Surveiller les releases Next.js
Mot de passe de démo public (mobile)	Faible	Vérifier qu'aucun compte prod ne l'utilise
Secret JWT dans l'historique git	Faible	Rotation effectuée (ancien secret inopérant)

13. Conclusion

La plateforme applique une défense en profondeur cohérente sur ses trois composants : authentification robuste, limitation de débit, en-têtes de sécurité, moindre privilège (conteneur non-root, utilisateur MySQL sans droits DDL, un seul port public), et une chaîne de livraison qui détecte automatiquement CVE et secrets.

Les écarts les plus significatifs concernent la conformité RGPD — traçabilité du consentement pour des données de santé, purge des journaux, portabilité — et sont hiérarchisés en tête des risques résiduels, garantissant leur suivi dans la durée.

Partie II — Plan de déploiement

1. Objet

Ce document décrit la stratégie de gestion du code source, l'intégration continue (CI) et le déploiement continu (CD) des trois applications de la plateforme CESIZen, ainsi que les environnements et la procédure de mise en production.

2. Gestion du code source

Hébergement. Trois dépôts GitHub sous l'organisation TxMy-Elu : `api_zen`, `cesizen_web`, `cesizen_mobile`.

Stratégie de branches. Modèle simplifié à deux branches longues :

Branche	Rôle	Protection
main	Production — code stable	PR obligatoire + checks verts (ruleset)
develop / dev	Intégration — travail courant	CI à chaque push
feature/** · fix/** · hotfix/**	Travail ponctuel	CI à chaque push

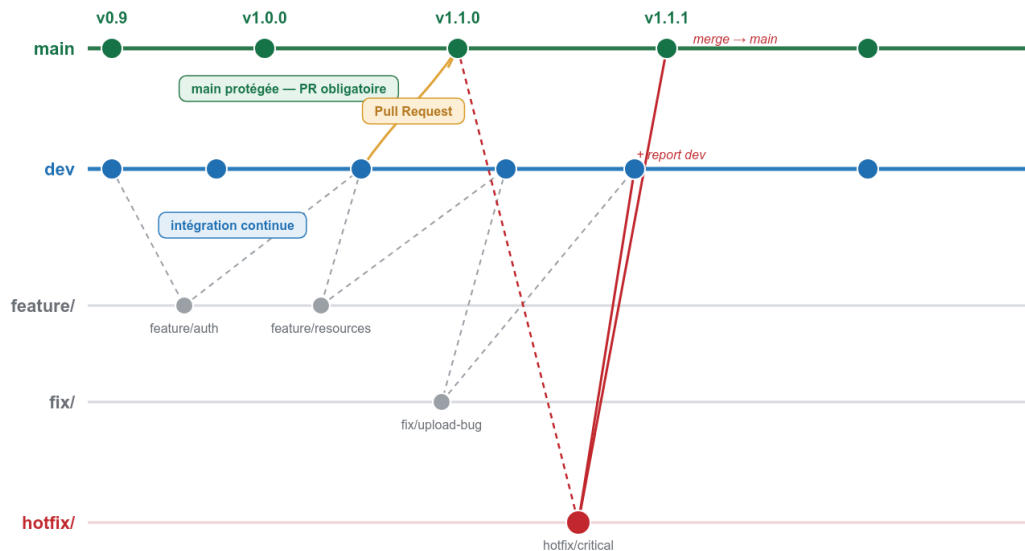
Convention de commits. Conventional Commits : `feat:`, `fix:`, `chore:`, `ci:`, `docs:`. Les tags de version suivent le SemVer (`v1.0.0`).

Protection de main. Chaque dépôt applique un ruleset GitHub : pull request obligatoire, interdiction de force-push et de suppression, et checks de statut requis (build/tests + scans de sécurité) avant tout merge.

Versionnement et releases. Chaque livraison sur `main` est taguée en SemVer (`git tag -a vX.Y.Z`). Le tag `v1.0.0` a été créé et poussé ; la CI se déclenche également sur les tags `v*.*.*`. **Écart identifié :** la création automatique d'une GitHub Release à chaque tag (notes de version, artefacts attachés) n'est pas encore en place — à ajouter via un workflow déclenché sur push : `tags`.

2.1 Modèle de branches

Le travail quotidien se fait sur `develop`. Les fonctionnalités et corrections partent de branches courtes, mergées dans `develop` par pull request. La mise en production passe par une pull request `develop` → `main`, protégée par le ruleset. Chaque livraison sur `main` est taguée.



2.2 Workflow quotidien

- 1. Se synchroniser : `git checkout develop && git pull`.
- 2. Créer une branche de travail : `git checkout -b feature/ma-fonctionnalite`.
- 3. Développer, committer en Conventional Commits, pousser — la CI se déclenche sur le push.
- 4. Ouvrir une pull request vers develop ; attendre les checks verts, puis merger.

2.3 Procédure de release (develop → main)

- 1. S'assurer que develop est à jour et vert en CI.
- 2. Ouvrir une pull request develop → main.
- 3. Le ruleset exige que les checks requis (build/tests + Gitleaks + Trivy) passent au vert.
- 4. Merger la PR — le déploiement se déclenche automatiquement.
- 5. Tagger la version : `git tag -a v1.1.0 -m "..." && git push origin v1.1.0`.

Point d'attention — protection de main. Le nom d'un check requis dans le ruleset correspond au **nom du job**, sans le préfixe du workflow (l'interface affiche pourtant « Security — CVE / Gitleaks »). Un nom mal recopié laisse le check en attente à l'infini et bloque le merge.

2.4 Correctif urgent (hotfix)

Un bug critique en production ne passe pas par **develop** : il serait ralenti par le travail en cours. On applique le flux **hotfix**, visible sur le schéma ci-dessus :

- 1. Créer une branche **hotfix/xxx** directement depuis **main**.
- 2. Corriger, ouvrir une pull request vers **main**, checks verts, merger.
- 3. Tagger le correctif (v1.0.1) — le déploiement se déclenche.
- 4. Reporter (backport) le correctif dans **develop** pour qu'il ne soit pas perdu à la prochaine release.

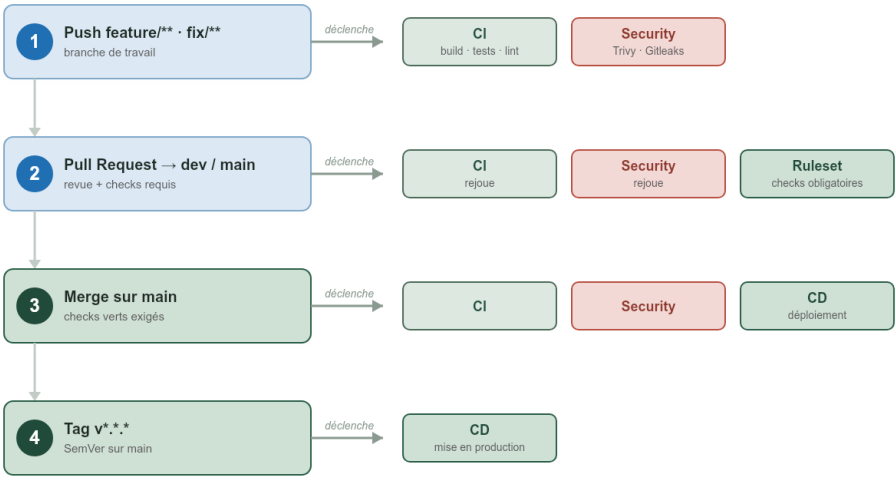
3. Intégration et déploiement continu (CI/CD)

Chaque dépôt possède **trois workflows GitHub Actions** distincts, volontairement séparés. Le tableau ci-dessous puis le détail par workflow décrivent l'ensemble.

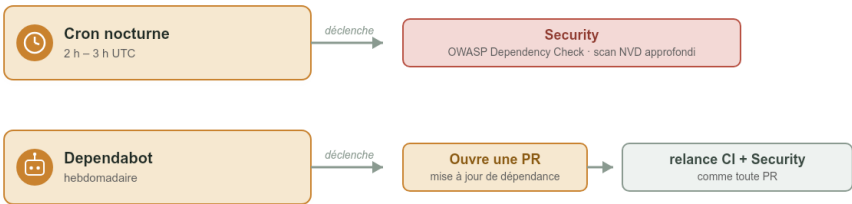
Workflow	Fichier	Déclencheurs
CI fonctionnelle	ci-*.yml	push (main, develop, feature/**, fix/**) · PR · tags v*.*.*
Sécurité	security-*.yml	push · PR · cron nocturne
Déploiement	deploy-*.yml	après CI verte sur main · déclenchement manuel

Ce qui se lance à chaque étape. Le schéma ci-dessous suit le workflow Git de haut en bas : pour chaque action (push d'une branche de travail, pull request, merge sur **main**, tag), il montre les workflows effectivement déclenchés. En bas figurent les deux déclencheurs planifiés — le cron nocturne et Dependabot — qui tournent indépendamment de toute action de l'équipe.

On suit le workflow Git — à chaque action, voici ce qui se lance



En parallèle — déclencheurs planifiés (indépendants du push)



À noter : Dependabot n'est pas un workflow au sens strict — c'est un service GitHub planifié (hebdomadaire) qui **ouvre des pull requests** de mise à jour. Ces PR relancent alors, comme n'importe quelle PR, la CI et le workflow de sécurité — ce qui valide automatiquement chaque montée de version avant tout merge.

3.1 CI fonctionnelle — build & tests

Vérifie que le code compile et que les tests passent, sur chaque push et chaque pull request.

Application	Étapes	Artefact
API (Maven)	mvn clean verify · JaCoCo (couverture) · SonarCloud · service MySQL	JAR + rapports de tests
Web (pnpm)	install --frozen-lockfile · lint (ESLint) · build Next.js	build de production
Mobile (Flutter)	flutter analyze · flutter test · build APK release	APK (clé debug)

3.2 Sécurité — CVE & secrets

Isolé de la CI pour qu'un scan lent ou une base CVE indisponible ne bloque jamais le build. En résumé : **Trivy** (CVE des dépendances, secrets, mauvaises configurations, image Docker côté API), **Gitleaks** (secrets dans l'historique git) et, sur l'API uniquement, **OWASP Dependency Check** (scan NVD approfondi, nocturne). Sur les dépôts publics, les résultats sont remontés dans l'onglet Security via des rapports SARIF.

3.3 Déploiement continu (CD)

Le déploiement n'est jamais lancé manuellement depuis une branche de travail : il n'intervient qu'**après un merge sur main** avec des checks verts (étape 3 du schéma ci-dessus), ou sur un **tag v*.*.*** pour une mise en production versionnée. Chaque application a sa cible : Vercel pour le web, image GHCR pour l'API, artefact APK pour le mobile — le détail figure au §4.

4. Déploiement par application

4.1 API — conteneur Docker sur GHCR

Image Docker multi-stage (Alpine, utilisateur non-root) construite et poussée sur le **GitHub Container Registry** après une CI verte sur `main`. Le job de déploiement SSH est **gated** par la variable `PROD_ENABLED` : il ne s'exécute que lorsqu'un serveur cible est disponible. En l'absence de VPS, la CD existe, reste non bloquante, et ne se déclenche pas — l'image est néanmoins prête à déployer.

4.2 Front — Vercel

Le déploiement est assuré par l'**intégration Git native de Vercel** : chaque push sur `main` est build et mis en production automatiquement. Un workflow GitHub Actions `deploy-web.yml` fournit en complément un **redéploiement manuel de secours** (déclenchement explicite + confirmation « PROD »), indépendant du tableau de bord Vercel — utile en cas d'incident.

4.3 Mobile — artefact APK

La CI produit un **APK release** signé avec la clé debug, publié comme artefact GitHub (rétention 7 jours). La publication sur les stores (Play Store / App Store) n'est pas automatisée dans le périmètre du projet ; l'APK constitue le livrable de déploiement.

5. Environnements

Deux environnements sont mis en œuvre : développement (local) et production. Un environnement de recette (staging) est conçu mais non déployé, faute d'infrastructure d'hébergement.

Paramètre	Développement	Staging (conçu)	Production
Déclenchement	Manuel (local)	Push sur develop	Push sur main
API	Docker local :8080	Instance dédiée	Image GHCR (gated)
Web	pnpm dev :3000	Preview Vercel	Vercel
Mobile	Émulateur 10.0.2.2:8080	APK de test	APK distribué
Base de données	MySQL Docker (cesizen_dev)	Base séparée	cesizen_prod — droits minimaux
Swagger	Activé	Activé	Désactivé
Logs	DEBUG	INFO	WARN
HTTPS	Non (HTTP local)	Oui	Oui (HSTS)
Données	Jeux de test	Fictives	Réelles

Ressources nécessaires par environnement. Chaque environnement mobilise des ressources distinctes (matériel, services managés, exécutants de CI/CD) dont le coût reste maîtrisé grâce aux offres gratuites et à l'infrastructure serverless.

Environnement	Hébergement / matériel	Services managés	Coût indicatif
Développement	Poste développeur — Docker Desktop (≈ 4 vCPU / 8 Go RAM)	Aucun (tout en local)	0 €
Staging (conçu)	VPS 2 vCPU / 4 Go / 40 Go SSD	—	≈ 5-10 €/mois
Production — API	VPS 2 vCPU / 4 Go / 80 Go SSD (Docker)	GitHub Container Registry (image)	≈ 10-15 €/mois

Environnement	Hébergement / matériel	Services managés	Coût indicatif
Production — Web	Serverless (aucun serveur à gérer)	Vercel (CDN, build) · Supabase (médias)	0 € (Hobby) → 20 €/mois (Pro)
Production — Mobile	Aucun serveur — artefact APK	GitHub Actions (build + artefact)	0 €

Ressources d'automatisation (CI/CD). Les workflows s'exécutent sur les **runners GitHub Actions** hébergés (ubuntu-latest), **gratuits et illimités sur dépôts publics**. Aucune infrastructure de build à maintenir. Le seul poste de coût récurrent réel du projet est l'**hébergement de l'API** (VPS) ; le web et le mobile n'engendrent aucun coût serveur.

Profils Spring. La bascule s'opère par `SPRING_PROFILES_ACTIVE : application-dev.properties` (logs verbeux, Swagger actif) et `application-prod.properties` (logs réduits, Swagger désactivé, ddl-auto=validate).

Variables d'environnement. Aucun secret n'est commité. Les valeurs sensibles vivent dans : le `.env` local (ignoré par git), les **secrets GitHub Actions** (identifiants BDD, JWT, token Vercel), les **secrets Dependabot** (coffre distinct — Dependabot n'a pas accès à ceux d'Actions), et les **variables d'environnement Vercel** (clés publiques Supabase, URL API).

6. Supervision du déploiement

La stack de supervision accompagne la mise en production et permet de valider le bon fonctionnement après déploiement.

Composant	Rôle	Port
Spring Boot Actuator	Expose /actuator/health et /actuator/prometheus	interne
Prometheus	Collecte des métriques (scrape 15 s)	9090 (interne)
Grafana	Tableaux de bord (JVM, HikariCP, Tomcat, HTTP)	3001
Loki + Promtail	Agrégation des journaux	3100 (interne)

Métriques suivies. Débit HTTP, latence p50/p95/p99, taux d'erreur, pool de connexions BDD (HikariCP), threads Tomcat, mémoire JVM.

Limite et action recommandée. Cette stack tourne sur le même hôte que l'API : si le serveur tombe, la supervision tombe avec lui. Une **sonde externe** (UptimeRobot ou équivalent) interrogeant /actuator/health depuis Internet, avec alerte e-mail, est nécessaire pour détecter une indisponibilité totale.

7. Procédure de mise en production

- 1. Ouvrir une pull request `develop` → `main`.
- 2. Attendre les checks verts (build, tests, scans de sécurité).
- 3. Merger la PR — le déploiement se déclenche (Vercel pour le web, image GHCR pour l'API).
- 4. Tagger la version sur `main` (`git tag -a vX.Y.Z`).

Rollback. Web : redéployer un déploiement Vercel antérieur (dashboard ou workflow manuel). API : re-taguer sur GHCR l'image précédente et relancer. Mobile : redistribuer l'APK de la version stable précédente.

8. Conclusion

La chaîne de livraison est automatisée de bout en bout et homogène sur les trois applications : intégration continue systématique, protection de la branche de production, et déploiement piloté par des déclencheurs explicites. Le principe directeur — aucun secret dans le code, aucun déploiement non maîtrisé — est appliqué sur l'ensemble du système.

Partie III — Dossier de maintenance

1. Objet

Ce dossier décrit l'organisation de la maintenance de la plateforme CESIZen : supervision, gestion des tickets, mises à jour préventives, veille, procédure de réponse à incident et types de maintenance avec leurs engagements de service.

2. Supervision et monitoring

Stack en place (API). Spring Boot Actuator expose `/actuator/health` et `/actuator/prometheus`. Une stack **Prometheus + Grafana + Loki** (fichier compose dédié) collecte les métriques JVM, HikariCP, Tomcat et HTTP (débit, latence p50/p95/p99, erreurs).

Limite connue. Cette stack tourne sur le même hôte que l'API : si le serveur tombe, la supervision tombe avec lui. Une **sonde externe** (UptimeRobot / Better Stack) interrogeant `/actuator/health` depuis l'extérieur est recommandée pour détecter une indisponibilité totale et alerter par e-mail.

Journalisation applicative. Deux journaux métier alimentent l'audit : `log_connexion` (chaque tentative de connexion, succès/échec, IP, horodatage) et `log_activite` (audit AOP de toutes les mutations CREATE/UPDATE/DELETE).

3. Gestion des tickets (GitHub Issues)

Chaque dépôt utilise GitHub Issues comme outil de ticketing, avec des modèles de saisie guidés.

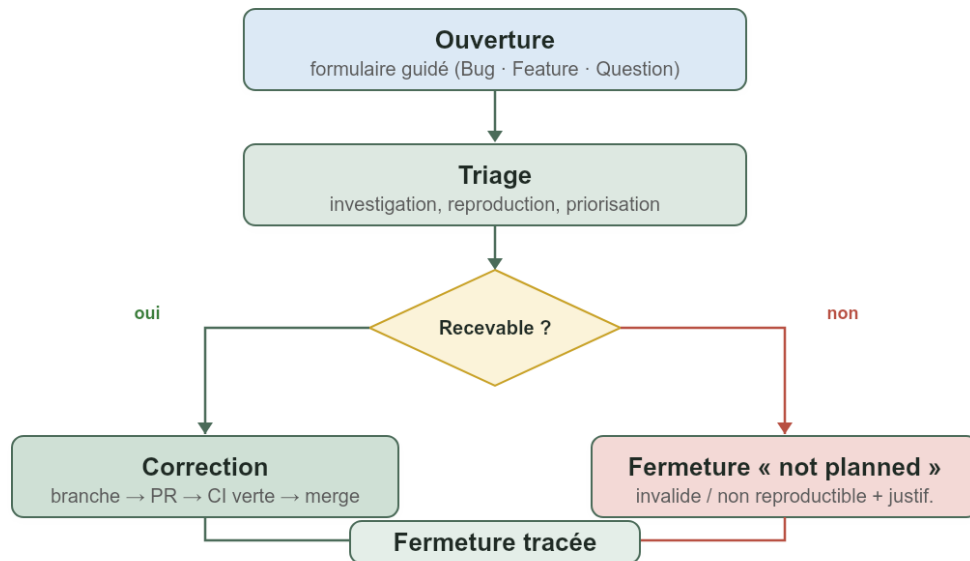
Modèles d'issue. Trois formulaires par dépôt (`.github/ISSUE_TEMPLATE/`) : Bug, Fonctionnalité, Question. Chacun impose des champs obligatoires (reproduction, criticité, environnement...). Les issues vierges sont désactivées ; les vulnérabilités sont redirigées vers un advisory privé.

Labels. Jeu homogène : `bug`, `enhancement`, `security`, `dependencies`, `documentation`, `question`, plus un label par composant.

Milestones. Chaque dépôt possède une milestone BL3 regroupant le reste-à-faire, avec barre de progression.

3.1 Traitement d'une issue

Chaque signalement suit un cycle de vie unique, du dépôt jusqu'à la fermeture tracée :



- **Ouverture.** L'auteur choisit un formulaire (Bug, Fonctionnalité, Question) ; les champs obligatoires garantissent un minimum d'information (reproduction, criticité, environnement). Le label correspondant est posé automatiquement.
- **Triage.** Un mainteneur reproduit, investigue en citant le code concerné, priorise et affecte une milestone.
- **Recevable → correction.** Une branche dédiée est créée, la correction passe par une pull request, la CI doit être verte, puis merge. L'issue est fermée « completed » et référencée par la PR.
- **Non recevable → fermeture « not planned ».** Une issue invalide, non reproductible ou déjà couverte est fermée avec une justification écrite — jamais laissée ouverte à vide. Une faille de sécurité, elle, ne passe pas par une issue publique mais par un advisory privé.

Exemples réels. Deux issues du projet illustrent ce cycle : un bump majeur cassant (react-day-picker 9→10), fermé puis neutralisé par une règle Dependabot ; et un bug d'authentification signalé, fermé « non reproductible » après vérification du code — dans les deux cas, une justification est tracée dans le fil.

4. Maintenance préventive — mises à jour

Dependabot. Configuré sur les trois dépôts, il ouvre chaque semaine des pull requests de mise à jour :

Dépôt	Écosystèmes	Particularités
api_zen	Maven, Docker, github-actions	Groupees jjwt et jackson (bumps solidaires)
cesizen_web	npm, github-actions	Groupees react/next ; ignore les majeures de react-day-picker (cassantes)
cesizen_mobile	pub, github-actions	—

Procédure de traitement. Une PR Dependabot est mergée uniquement si la CI est verte. Une montée majeure cassante (ex : react-day-picker 9→10) est fermée et, si récurrente, neutralisée par une règle `ignore` dans `dependabot.yml`. Les artefacts d'un même BOM sont groupés pour éviter les bumps partiels dangereux.

Scans continus. Trivy et Gitleaks tournent à chaque push et chaque nuit ; OWASP Dependency Check effectue un scan NVD approfondi nocturne sur l'API. Toute CVE CRITICAL/HIGH corrigible fait échouer la CI.

5. Veille technologique

La veille s'appuie sur des sources ciblées : spring.io/blog et les GitHub Releases (backend), nextjs.org/blog et le changelog Vercel (front), flutter.dev et pub.dev (mobile), et pour la sécurité **OWASP**, **CVE Mitre** et la base d'avis GitHub. Le suivi passe par l'abonnement (Watch) aux dépôts des dépendances critiques et par Dependabot, qui matérialise automatiquement les nouvelles versions. Trois fiches de veille détaillées accompagnent ce dossier.

6. Procédure de réponse à incident

- **Détection.** Alerte Dependabot, échec de scan sécurité, logs anormaux (log_connexion / log_activite), ou signalement utilisateur via une issue.
- **Isolation.** Couper le service concerné si nécessaire ; en cas de compromission de token, rotation du secret JWT (invalide tous les tokens).
- **Analyse.** Examiner les journaux d'activité et de connexion pour délimiter l'impact.
- **Correction.** Patch → pull request → CI verte → redéploiement.
- **Communication.** Notifier les utilisateurs si des données personnelles sont concernées.
- **RGPD.** Notification à la CNIL sous 72 h en cas de violation de données personnelles.

7. Types de maintenance et engagements

7.1 Maintenance préventive

Anticiper les défaillances avant qu'elles ne surviennent. Incluse dans le forfait de maintenance.

- Mises à jour hebdomadaires des dépendances (Dependabot) sur les trois dépôts.
- Scans de sécurité nocturnes (Trivy, Gitleaks, OWASP Dependency Check).
- Revue mensuelle des journaux de connexion à la recherche d'anomalies.
- Vérification de la disponibilité via la sonde externe.
- Audit SonarCloud trimestriel.

7.2 Maintenance corrective

Criticité	Définition	Prise en charge (SLA)	Résolution visée
Bloquante	Service indisponible ou faille exploitable	4 heures ouvrées	24 heures
Majeure	Fonctionnalité inutilisable, sans contournement	1 jour ouvré	72 heures
Mineure	Anomalie avec contournement possible	3 jours ouvrés	Prochaine version
Cosmétique	Défaut d'affichage sans impact fonctionnel	—	Selon disponibilité

Dans le périmètre : bugs introduits par les développements réalisés sur CESIZen.

Hors périmètre : pannes de l'hébergeur, failles de composants tiers non encore corrigées en amont, indisponibilités de services externes (Vercel, Supabase), et toute anomalie causée par une modification effectuée par le client.

7.3 Maintenance évolutive

Ajout de fonctionnalités ou adaptation à un nouveau besoin. Facturée au temps passé, hors forfait.

Élément	Valeur indicative
Tarif journalier moyen (TJM) — développeur	450 € HT / jour
Tarif journalier moyen (TJM) — chef de projet	600 € HT / jour
Unité de facturation minimale	0,5 jour
Forfait maintenance préventive + corrective	350 € HT / mois

Mode de contractualisation.

- 1. Le besoin est formalisé par une issue GitHub (modèle « Fonctionnalité »), avec ses critères d'acceptation.
- 2. Un chiffrage en jours-homme est produit et soumis au client.
- 3. Après validation écrite (bon de commande), la tâche est affectée à une milestone.
- 4. La livraison suit le processus standard : branche dédiée, pull request, CI verte, revue, merge, déploiement.
- 5. La recette est prononcée par le client sur la base des critères d'acceptation de l'issue.

Le contrat de maintenance est conclu pour une durée de **12 mois**, reconductible tacitement, résiliable avec un préavis de 2 mois. Les évolutions sont priorisées conjointement lors d'un comité de pilotage trimestriel.

Les montants ci-dessus sont donnés à titre d'hypothèse pédagogique et n'engagent aucune partie.

8. Conclusion

La maintenance de CESIZen repose sur l'automatisation (Dependabot, scans, CI) et sur une traçabilité complète (issues, milestones, journaux d'audit). La supervision est opérationnelle, avec une amélioration identifiée — la sonde externe — pour couvrir le cas de l'indisponibilité totale. Les procédures d'incident et les engagements de service sont documentés et reproductibles.

Partie IV — Veille technologique

Méthodologie

La veille est structurée autour de sources officielles et de bases de vulnérabilités, matérialisée dans le projet par Dependabot et les scans Trivy/OWASP. Chaque fiche suit la même trame : **source datée, résumé, impact sur CESIZen, action prise**. Les trois sujets retenus correspondent à des événements réels rencontrés pendant le projet.

Fiche 1 — Vulnérabilités Jackson (CVE-2026-54512 / 54513)

Champ	Contenu
Sujet	Désérialisation Jackson (jackson-databind)
Source	Base d'avis GitHub / NVD — remontée par Trivy en CI, juillet 2026
Références	CVE-2026-54512, CVE-2026-54513
Sévérité	CRITICAL / HIGH

Résumé

Deux vulnérabilités affectant jackson-databind, bibliothèque de sérialisation JSON transitive de Spring Boot. Exploitable lors de la désérialisation de données non fiables, elles peuvent conduire à un comportement dangereux côté serveur.

Impact sur CESIZen

L'API expose des endpoints REST qui désérialisent du JSON entrant. La version de Jackson héritée du parent Spring Boot était vulnérable. Le scan Trivy de la CI a fait **échouer le build**, révélant la CVE avant toute mise en production.

Action prise

Forçage des versions corrigées par import de BOM dans `pom.xml` : `com.fasterxml.jackson:jackson-bom:2.21.4` et `tools.jackson:jackson-bom:3.1.4` en `<dependencyManagement>`. Point technique notable : un `<dependencyManagement>` déclaré dans le projet **prime sur le BOM du parent** Spring Boot — seul moyen propre de forcer une version transitive sans figer chaque dépendance.

Fiche 2 — Spring Boot 4 & Spring Security 6

Champ	Contenu
Sujet	Montée de version majeure du socle backend
Source	spring.io/blog , notes de version Spring Boot 4.0.x — 2026
Nature	Évolution de framework

Résumé

Spring Boot 4 s'appuie sur Spring Framework 7 et Spring Security 6, avec Hibernate 7. Plusieurs changements de comportement impactent la configuration : la détection automatique du dialect Hibernate disparaît, et l'API de configuration des en-têtes de sécurité évolue.

Impact sur CESIZen

- Hibernate 7 ne détecte plus le dialect : ajout explicite de `spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect`.
- Spring Security 6 permet de configurer nativement les en-têtes (`permissionsPolicy`, `contentSecurityPolicy`) — exploité pour le durcissement HTTP.
- Tomcat embarqué mis à jour : occasion de forcer 11.0.22 pour corriger des CVE.

Action prise

Configuration adaptée et testée ; en-têtes de sécurité posés via la nouvelle API. La montée de version, loin d'être subie, a été l'occasion de renforcer la posture de sécurité (en-têtes natifs, Tomcat récent).

Fiche 3 — Vulnérabilité Next.js 16 (contournement de middleware)

Champ	Contenu
Sujet	Contournement de middleware Next.js
Source	Base d'avis GitHub — détectée par pnpm audit / Trivy, juillet 2026
Références	GHSA-36qx-fr4f-26g5 (et apparentées)
Sévérité	HIGH — corrigée en 16.2.5+

Résumé

Next.js 16.2.2 (version alors utilisée par le front) portait plusieurs vulnérabilités de sévérité HIGH, dont un contournement de middleware permettant, dans certaines configurations, d'atteindre des routes normalement protégées. Corrigé à partir de la 16.2.5.

Impact sur CESIZen

Le front Next.js s'appuie sur le routage pour protéger l'espace d'administration. Le scan de sécurité de la CI web a remonté **huit vulnérabilités HIGH** dans Next.js, dont le step Trivy bloquant aurait fait échouer toute livraison.

Action prise

Montée de Next.js en **16.2.10** (et de l'outillage associé : eslint-config-next, postcss), build et tests revérifiés. Les huit CVE HIGH ont disparu. Une CVE moderate résiduelle (postcss embarqué dans le toolchain interne de Next) subsiste sous le seuil bloquant et reste sous surveillance. Cet épisode illustre la valeur d'un scan de dépendances **bloquant** en intégration continue.

Synthèse

Les trois fiches montrent une veille outillée : les vulnérabilités ne sont pas découvertes manuellement mais remontées automatiquement par la chaîne CI (Trivy, audit npm, OWASP), puis traitées par mise à jour ciblée. La veille sur les frameworks (Spring Boot 4, Next.js 16) permet d'anticiper les changements de comportement et de transformer chaque montée de version en occasion de durcissement.